

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА  
Факультет математики, фізики та інформаційних технологій  
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА  
з освітнього компоненту «Програмування»  
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ  
«Спрощений варіант №20»

Студента 1 курсу, денна форма навчання  
спеціальність F7 «Комп'ютерна інженерія»

Чебана Вадима Сергійовича

Керівник: к.т.н., доц. Шестопалов С.В.

Національна шкала: \_\_\_\_\_

Кількість балів: \_\_\_\_\_

ECTS: \_\_\_\_\_

Дата захисту: \_\_\_\_\_

Члени комісії \_\_\_\_\_ Антоненко О.С.

(підпис) (прізвище та ініціали)

\_\_\_\_\_ Шестопалов С.В.

(підпис) (прізвище та ініціали)

\_\_\_\_\_ \_\_\_\_\_

(підпис) (прізвище та ініціали)

## ЗМІСТ

|                                 |    |
|---------------------------------|----|
| ВСТУП.....                      | 3  |
| ПОСТАНОВКА ЗАВДАННЯ.....        | 4  |
| 1. ТЕОРЕТИЧНА ЧАСТИНА.....      | 6  |
| 1.1 Поняття ООП.....            | 6  |
| 1.2 Опис об'єктної частини..... | 7  |
| 2. ПРАКТИЧНА ЧАСТИНА.....       | 9  |
| 2.1 Опис класів.....            | 9  |
| 2.2 Інтерфейс.....              | 12 |
| ВИСНОВОК.....                   | 15 |
| СПИСОК ЛІТЕРАТУРИ.....          | 16 |

## ВСТУП

Мета роботи – створити ієрархію класів за спрощеним варіантом № 20 та застосувати її для розв’язання прикладної задачі. Під час виконання роботи використовуються основні принципи об’єктно-орієнтованого програмування: інкапсуляція, успадкування та поліморфізм. Згідно із завданням потрібно реалізувати систему взаємопов’язаних класів, що описують людину та її характеристики.

Для цього необхідно розробити базовий клас Person та його спадкоємців, створити конструктори, закриті поля та методи для роботи з об’єктами. У програмі слід реалізувати гнучкий сценарій роботи через текстове меню для додавання, видалення та виведення інформації про об’єкти.

## ПОСТАНОВКА ЗАВДАННЯ

1) Взяти клас Person із четвертої лабораторної (див. таблицю Розробити клас Person, який описує людину, з наступними можливостями:

~ полями та методами зазначеними у таблиці (за варіантами):

- a) ПІБ fullName;
- b) зріст height;
- c) вага weight;
- d) країна проживання country;
- e) номер телефону phoneNumber;
- f) перевірити чи має зріст понад 200 см?
- g) чи живе у Польщі?
- h) Якщо номер телефону починається з 0, додати спочатку номер телефону +38.

~ створити конструктор без параметрів та конструктор з параметрами.

~ створити метод «змінити номер телефону»

~ при створенні людини (у конструкторі) перевіряти зріст та вагу

~ реалізувати вставку людини у потік виведення (роздруківку), або метод to\_string(), або метод print().

2) Реалізувати два спадкоємці класу Person (див. таблицю 2 за варіантами). У спадкоємцях перевизначити оператор вставки в потік або, відповідно, або метод to\_string(), або метод print().

- ~ Студент: додаткові поля: ВНЗ, Курс, Факультет, оцінки за іспити (вектор), метод: перевести на наступний курс, якщо всі оцінки  $\geq 60$  балів, повернути стипендію (звичайна, якщо середня оцінка не менша за «добре», підвищена, якщо всі оцінки "відмінно"). Також реалізувати виведення всіх студентів, які отримують стипендію.
- ~ Робітник: додаткові поля: завод, посада, оклад. Метод збільшити оклад, який збільшує оклад робітника, залежно від посади. Також реалізувати метод – змінити посаду.

3) Створити програмну систему з можливістю додавання, видалення, виведення інформації про екземпляри типу Person та його спадкоємців (додатково: пошук). Використовувати вектор вказівників на Person (як менш оптимальний варіант припустимо використовувати кілька векторів елементів відповідних класів).

Номер варіанта – це номер студента у списку групи – в даному випадку це варіант 20.

В основній програмі бажано реалізувати гнучкий сценарій роботи програми, наприклад, через текстове меню, щоб користувач міг вибирати свої чергові дії у довільному порядку.

## 1. ТЕОРЕТИЧНА ЧАСТИНА

### 1.1 Поняття ООП

Об'єктно-орієнтоване програмування є методологією розробки, в якій структура програми базується на взаємодії об'єктів. Кожен такий об'єкт є окремим екземпляром конкретного класу, що входять до загальної ієрархії наслідування [1]. Розробник спочатку описує клас (шаблон), а вже під час роботи програми на його основі створюються реальні об'єкти (екземпляри). Існуючі класи можна використовувати як фундамент для нових, які розширюють їхній функціонал, завдяки чому і формується ієрархічна структура.

Будь-яка парадигма програмування спирається на власну теоретичну базу. В основі ОО-стилю лежить об'єктна модель, побудована на трьох ключових концепціях [2,3]:

- а) інкапсуляція;
- б) поліморфізм;
- с) наслідування.

Інкапсуляція – це концепція, яка об'єднує дані та функції (методи) для їхньої обробки в один елемент, захищаючи їх від несанкціонованого зовнішнього доступу або некоректного використання. Таке поєднання компонентів власне і формує об'єкт [1].

Наслідування є процесом, що дозволяє одному класу переймати характеристики та поведінку іншого (базового) класу, доповнюючи їх власними унікальними властивостями, притаманними лише йому [1].

Поліморфізм – принцип, який дає змогу застосовувати однакове ім'я для виконання схожих за змістом, але різних за технічною реалізацією завдань (тобто використання одного інтерфейсу для загального класу дій). На практиці

це виражається у здатності об'єкта автоматично обирати потрібний внутрішній метод залежно від типу вхідних даних [1].

## 1.2 Опис об'єктної частини

Програма пропонує інформаційну систему обліку та керування даними, яка побудована на базі об'єктно-орієнтованої ієрархії із застосуванням принципів успадкування, інкапсуляції та поліморфізму. Основою архітектури є базовий клас `Person`, який описує людину та містить її загальні анкетні характеристики: прізвище, ім'я, країну проживання, а також контактні й фізичні параметри. Можливість виконання тих чи інших дій у програмі або коректність збереження сутностей безпосередньо залежить від вбудованої валідації даних: властивості класу контролюють, щоб зріст та вага не набували від'ємних значень, а імена не залишалися порожніми. Для забезпечення динамічного керування та підтримки поліморфної поведінки всі екземпляри базового та похідних класів зберігаються в єдиній колекції – динамічному списку об'єктів базового типу (`List<Person>`).

Звичайний користувач системи взаємодіє з базовим класом `Person` та двома його прямими спадкоємцями – `Student` та `Employee`. Спадкоємці повністю перебирають на себе базовий функціонал, але адаптують його під конкретні категорії осіб:

а) Клас `Student` (Студент) додатково оперує логікою навчального процесу, авторизується за назвою університету та поточним курсом навчання.

б) Клас `Employee` (Працівник) отримує доступ до розширених полів професійної діяльності, де зберігаються дані про компанію, (місце роботи) займану посаду, та оклад.

Усі три класи перевизначають базовий метод `ToString()`, що дозволяє автоматично виводити повну інформацію про об'єкт у потік відповідно до його фактичного типу.

Клас Menu та головна програма реалізують сценарій роботи через текстове меню, щоб користувач міг вибирати свої чергові дії у довільному порядку. Меню пропонує користувачеві чітко розподілений функціонал: перегляд повної бази, додавання нових людей (відповідно до трьох категорій) із заповненням усіх властивостей, а також видалення об'єктів і т.д.. Окремим важливим елементом є режим пошуку за прізвищем, який не лише знаходить збіги у списку database, а й проводить інтегрований аналіз характеристик людини – автоматично перевіряє за допомогою методів IsHeightOver200() та LivesInPoland(), чи перевищує зріст особи 200 см і чи проживає вона на території Польщі.

## 2. ПРАКТИЧНА ЧАСТИНА

### 2.1 Опис класів

#### 1) Клас «Person»:

a) Приватні поля: firstName, lastName, height, weight, country, phoneNumber.

b) Конструктори:

- ~ Конструктор без параметрів.
- ~ Конструктор з параметрами.

c) Методи:

- ~ IsHeightOver200 – аналіз зросту, повертає "Так", якщо зріст особи перевищує 200 см.
- ~ LivesInPoland – аналіз географічного положення, повертає "Так", якщо країна проживання особи дорівнює "Польща" або "Poland".
- ~ CheckAndFixPhoneNumber – внутрішній метод автоматичного виправлення формату номера: якщо телефон починається з "0", додає "+38".
- ~ Перевизначення ToString – повертає відформатований інформаційний рядок про базові дані людини.

#### 2) Клас «Student», нащадок класу «Person»:

a) Властивості (поля): University, CourseYear.

b) Конструктори:

- ~ Конструктор без параметрів.
- ~ Конструктор з параметрами.

c) Методи:

- ~ Перевизначення ToString – повертає повну інформацію про об'єкт, доповнюючи базові дані людини академічними атрибутами студента.
- ~ GetScholarshipType() – метод який аналізує яка повинна бути стипендія у студента.

3) Клас «Employee», нащадок класу «Person»:

a) Властивості (поля): Company, Position, Salary.

b) Конструктори:

~ Конструктор без параметрів.

~ Конструктор з параметрами

c) Методи:

~ Перевизначення ToString – повертає сформований рядок з інформацією про об'єкт, поєднуючи особисті дані з назвою компанії та займаною посадою.

~ ChangePosition(string newPosition) – змінює посаду працівника.

~ IncreaseSalary() – збільшує оклад працівника (в залежності від посади).

4) Статичний клас «Menu»:

a) Приватні поля: database

b) Методи:

~ InitDatabase – заповнює колекцію database початковими тестовими записами для кожного типу об'єктів (Person, Student, Employee).

~ PrintAll – перевіряє базу на наявність елементів та виводить у консоль список усіх зареєстрованих людей, використовуючи поліморфний виклик методу ToString().

~ AddPerson – універсальний інтерактивний метод, який зчитує дані з консолі та додає до списку новий об'єкт (Person або одного з нащадків) залежно від переданого типу (type).

~ RemovePerson – виконує пошук за прізвищем користувача серед елементів колекції та, у разі успішного збігу, повністю видаляє знайдений запис із бази.

~ SearchPerson – здійснює пошук усіх людей із заданим прізвищем, виводить їхні повні картки та запускає інтегровані методи аналізу щодо зросту (IsHeightOver200) і місця проживання (LivesInPoland).

- ~ PrintStudentsWithScholarship() – виводить студентів які отримують стипендію.
- ~ ChangeEmployeePosition() – змінює посаду працівника.
- ~ IncreaseEmployeeSalary() – змінює оклад працівника(залежить від посади).

## 2.2 Інтерфейс

Готовий проєкт розміщено в [4]. Під час запуску програми користувач бачить головне вікно з текстовим меню (Рис. 2.1).

```
=== ГОЛОВНЕ МЕНЮ ===  
1. Вивести всіх людей  
2. Додати звичайну Людину  
3. Додати Студента  
4. Додати Працівника  
5. Видалити за прізвищем  
6. Знайти людину за прізвищем  
7. Вивести студентів, які отримують стипендію  
8. Змінити посаду працівника  
9. Збільшити оклад працівнику  
0. Вихід  
Оберіть дію: |
```

Рисунок 2.1 – Головне меню програми

Цей екран є центральним вузлом управління системою. Користувачеві пропонується ввести цифру від 0 до 9 для вибору необхідної операції. Якщо обрати пункт «1», програма звертається до методу Menu.PrintAll() та виводить на екран повний список усіх людей, що наразі зареєстровані в базі даних (Рис. 2.2).

```
Список записів:  
[Людина] Прізвище: Шевченко, Ім'я: Тарас, Зріст: 185см, Вага: 80кг, Країна: Україна, Тел: +380991234567  
[Студент] Прізвище: Франко, Ім'я: Іван, Зріст: 175см, ВНЗ: Мечникова, Курс: 3, Факультет: Філологічний, Оцінки: [95, 90,  
88], Стипендія: звичайна  
[Робітник] Прізвище: Міцкевич, Ім'я: Ілля, Зріст: 205см, Країна: Польща, Завод: IT-Corp, Посада: Розробник, Оклад: 45000  
,00 грн
```

Рисунок 2.2 – Виведення списку всіх людей

Для розширення бази даних користувач може скористатися пунктами меню 2, 3 або 4. Залежно від вибору, система очікує послідовне введення загальних анкетних характеристик особи, а також специфічних полів для профільних класів (Рис. 2.3).

```

Введіть прізвище: Чебан
Введіть ім'я: Вадим
Введіть зріст (см): 190
Введіть вагу (кг): 80
Введіть країну проживання: Україна
Введіть номер телефону: 0955825423
Введіть ВНЗ: Мечникова
Введіть курс: 1
Введіть факультет: КІ
Введіть оцінки через пробіл: 95 98 99
Запис успішно додано!

```

Рисунок 2.3 – Вікно додавання нового Студента

У разі вибору пункту «5» користувач може видалити будь-яку особу із загальної колекції. Для цього система просить ввести прізвище. (Рис. 2.4).

```

Оберіть дію: 5
-----
Введіть прізвище для видалення: Чебан
Запис видалено.

```

Рисунок 2.4 – Процес видалення запису за прізвищем

Пункт головного меню «6» активує режим пошуку. Користувач вводить прізвище особи, після чого програма не лише виводить повну картку знайденого об'єкта завдяки поліморфному методу ToString(), а й автоматично проводить аналітичну перевірку фізичних та географічних параметрів за допомогою методів IsHeightOver200() та LivesInPoland() (Рис. 2.5).

```

Введіть прізвище для пошуку: Чебан
Знайдено 1 запис(ів):
[Студент] Прізвище: Чебан, Ім'я: Вадим, Зріст: 190см, ВНЗ: Мечникова, Курс: 1, Факультет: КІ, Оцінки: [95, 98, 99], Стипендія: підвищена
Зріст > 200 см? Ні
Живе у Польщі? Ні

```

Рисунок 2.5 – Вікно пошуку та аналізу особи за прізвищем

Пункт під номером «7» виводить усіх студентів які отримують стипендію. Якщо у студента усі предмети закриті с балами більше 90, то він отримує підвищену стипендію (Рис. 2.7).

```
Оберіть дію: 7
-----
Студенти, які отримують стипендію:
[Студент] Прізвище: Франко, Ім'я: Іван, Зріст: 175см, ВНЗ: Мечникова, Курс: 3, Факультет: Філологічний, Оцінки: [95, 90, 88], Стипендія: звичайна
[Студент] Прізвище: Чебан, Ім'я: Вадим, Зріст: 190см, ВНЗ: Мечникова, Курс: 1, Факультет: КІ, Оцінки: [95, 98, 99], Стипендія: підвищена
```

Рисунок 2.7 – Виведення списку студентів, які отримують стипендію

Пункт головного меню «8» дає змогу змінити посаду працівника (Рис. 2.9).

```
Оберіть дію: 8
-----
Введіть прізвище працівника: Міцкевич
Поточна посада: Розробник. Введіть нову посаду: менеджер
Посаду успішно змінено!
```

Рисунок 2.8 – Зміна посади працівника

Пункт номер «9» дає змогу підвищити оклад працівнику, в залежності від його посади (Рис. 2.9).

```
Оберіть дію: 9
-----
Введіть прізвище працівника: Міцкевич
Оклад успішно збільшено! Новий оклад: 54000,00 грн.
```

Рисунок 2.9 – Зміна окладу працівника

Для завершення роботи та закриття консольного додатка користувачеві достатньо ввести символ «0».

## ВИСНОВОК

Під час виконання курсової роботи було розроблено консольну програму на мові програмування C# для автоматизації обліку даних про різні категорії людей. Програма побудована з використанням об'єктно-орієнтованого підходу, що дозволило створити гнучку та масштабовану архітектуру.

У ході роботи було реалізовано ієрархію класів, де базовий клас Person містить загальні антропометричні та контактні дані, а похідні класи Student та Employee розширюють цей функціонал специфічними властивостями (місце навчання, посада тощо). Особливу увагу приділено принципу інкапсуляції: доступ до полів класу здійснюється через властивості з інтегрованою валідацією даних, що запобігає появі логічних помилок (наприклад, введення від'ємного зросту чи порожнього імені).

Для управління базою даних використано динамічну колекцію List<Person>, яка завдяки механізму поліморфізму дозволяє зберігати об'єкти різних типів в одному списку та коректно опрацьовувати їх за допомогою віртуальних методів. Розроблений інтерфейс у вигляді текстового меню забезпечує зручну взаємодію з користувачем, дозволяючи виконувати такі операції:

- a) додавання нових записів різних типів;
- b) перегляд повної бази даних;
- c) видалення та пошук інформації за прізвищем;
- d) проведення автоматизованого аналізу параметрів (перевірка країни проживання та фізичних показників).

Програма пройшла тестування на різних наборах даних, продемонструвавши стабільну роботу та коректну обробку виняткових ситуацій за допомогою блоків try-catch. Результати роботи підтвердили

ефективність використання об'єктно-орієнтованих технологій для створення прикладного програмного забезпечення.

### СПИСОК ЛІТЕРАТУРИ

1. Грицюк Ю. І., Рак Т. Є. Об'єктно-орієнтоване програмування мовою С#. Навчальний посібник. – Львів: Вид-во ЛДУ БЖД, 2024. – 360 с.
2. Фрімен А. С# для професіоналів. Тонкощі програмування / А. Фрімен. – Київ: Діалектика, 2025. – 850 с.
3. Специфікація мови С#: класи, структури та записи [Електронний ресурс] // Microsoft Learn. – Режим доступу: <https://learn.microsoft.com/uk-ua/dotnet/csharp/fundamentals/types/classes>
4. Репозиторій GitHub на курсовий – Режим доступу: <https://github.com/Nineteeeeen/Kursach>

## Додаток А. Діаграма класів

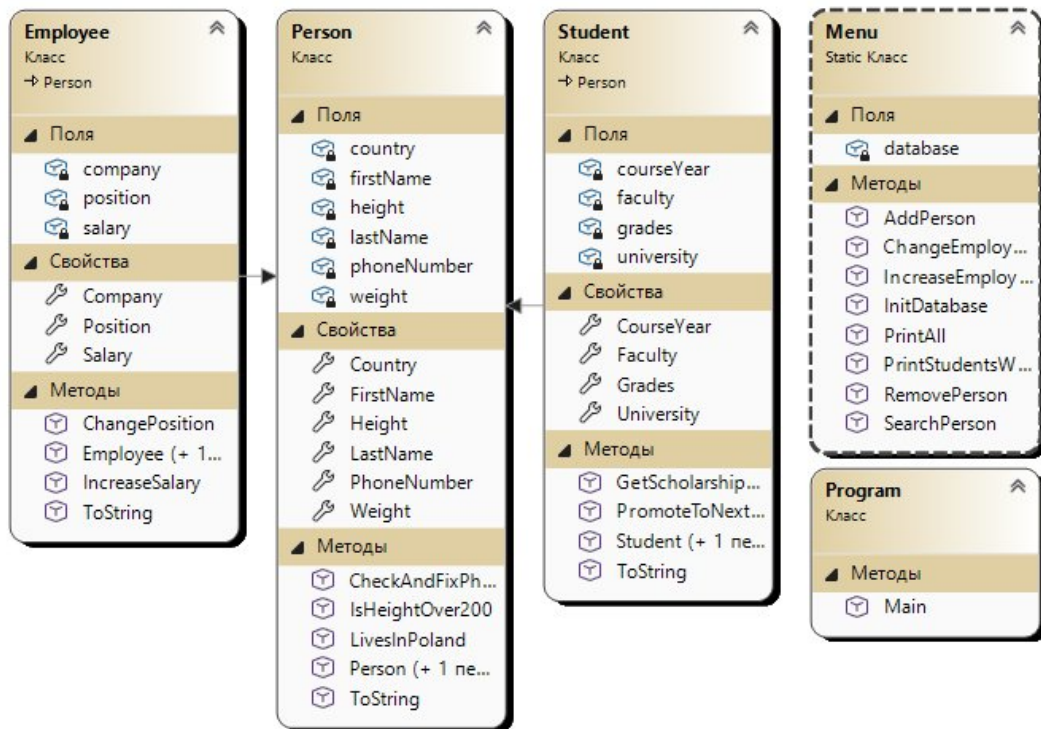


Рисунок А1. – Діаграма класів

## Додаток Б. Код класу

```

using System;
using System.Collections.Generic;
using System.Linq;

namespace ConsoleApp1
{
    public static class Menu
    {
        private static List<Person> database = new List<Person>();

        public static void InitDatabase()
        {
            database.Add(new Person("Шевченко", "Тарас", 185, 80, "Україна", "+380991234567"));
            database.Add(new Student("Франко", "Іван", 175, 70, "Україна", "+380501112233", "Мечникова", 3,
"Філологічний", new List<int> { 95, 90, 88 }));
            database.Add(new Employee("Мицкевич", "Ілля", 205, 90, "Польща", "+380679998877", "IT-Corp",
"Розробник", 45000.0));
        }

        public static void PrintAll()
        {
            if (database.Count == 0)
            {
                Console.WriteLine("База даних порожня.");
                return;
            }

            Console.WriteLine("Список записів:");
            foreach (Person person in database)
            {
                Console.WriteLine(person.ToString());
            }
        }

        public static void AddPerson(int type)
        {
            Console.Write("Введіть прізвище: ");
            string fName = Console.ReadLine() ?? "";
            Console.Write("Введіть ім'я: ");
            string lName = Console.ReadLine() ?? "";
            Console.Write("Введіть зріст (см): ");
            int height = int.Parse(Console.ReadLine() ?? "0");
            Console.Write("Введіть вагу (кг): ");
            int weight = int.Parse(Console.ReadLine() ?? "0");
            Console.Write("Введіть країну проживання: ");
            string country = Console.ReadLine() ?? "";
            Console.Write("Введіть номер телефону: ");
            string phone = Console.ReadLine() ?? "";

            if (type == 1)
            {
                database.Add(new Person(fName, lName, height, weight, country, phone));
            }
            else if (type == 2)
            {
                Console.Write("Введіть ВНЗ: ");
                string uni = Console.ReadLine() ?? "";
                Console.Write("Введіть курс: ");
                int course = int.Parse(Console.ReadLine() ?? "1");
                Console.Write("Введіть факультет: ");
            }
        }
    }
}

```

Лістинг Б.1 – Клас Menu, аркуш 1

```

string faculty = Console.ReadLine() ?? "";
Console.Write("Введіть оцінки через пробіл: ");
string gradesInput = Console.ReadLine() ?? "";

    List<int> grades = new List<int>();
    foreach (string g in gradesInput.Split(' '))
    {
        if (int.TryParse(g, out int grade))
            grades.Add(grade);
    }

    database.Add(new Student(fName, lName, height, weight, country, phone, uni, course, faculty, grades));
}
else if (type == 3)
{
    Console.Write("Введіть Завод/Компанію: ");
    string comp = Console.ReadLine() ?? "";
    Console.Write("Введіть Посаду: ");
    string pos = Console.ReadLine() ?? "";
    Console.Write("Введіть Оклад: ");
    double salary = double.Parse(Console.ReadLine() ?? "0");

    database.Add(new Employee(fName, lName, height, weight, country, phone, comp, pos, salary));
}

Console.WriteLine("Запис успішно додано!");
}

public static void RemovePerson()
{
    Console.Write("Введіть прізвище для видалення: ");
    string targetName = Console.ReadLine() ?? "";

    Person personToRemove = null;

    foreach (Person p in database)
    {
        if (p.FirstName.ToLower() == targetName.ToLower())
        {
            personToRemove = p;
            break;
        }
    }

    if (personToRemove != null)
    {
        database.Remove(personToRemove);
        Console.WriteLine("Запис видалено.");
    }
    else
    {
        Console.WriteLine("Нікого не знайдено.");
    }
}

public static void SearchPerson()
{
    Console.Write("Введіть прізвище для пошуку: ");
    string targetName = Console.ReadLine() ?? "";

    List<Person> results = new List<Person>();

```

```

foreach (Person p in database)
{
    if (p.FirstName.ToLower() == targetName.ToLower())
    {
        results.Add(p);
    }
}

if (results.Count > 0)
{
    Console.WriteLine($"Знайдено {results.Count} запис(ів):");
    foreach (Person p in results)
    {
        Console.WriteLine(p.ToString());
        Console.WriteLine($" Зріст > 200 см? {p.IsHeightOver200()}");
        Console.WriteLine($" Живе у Польщі? {p.LivesInPoland()}");
    }
}
else
{
    Console.WriteLine("Нікого не знайдено.");
}
}

public static void PrintStudentsWithScholarship()
{
    List<Student> studentsWithScholarship = new List<Student>();

    foreach (Person p in database)
    {
        if (p is Student student)
        {
            if (student.GetScholarshipType() != "немає")
            {
                studentsWithScholarship.Add(student);
            }
        }
    }

    if (studentsWithScholarship.Count == 0)
    {
        Console.WriteLine("Студентів, які отримують стипендію, не знайдено.");
        return;
    }

    Console.WriteLine("Студенти, які отримують стипендію:");
    foreach (Student student in studentsWithScholarship)
    {
        Console.WriteLine(student.ToString());
    }
}

public static void ChangeEmployeePosition()
{
    Console.Write("Введіть прізвище працівника: ");
    string targetName = Console.ReadLine() ?? "";

    foreach (Person p in database)
    {

```

```

if (p is Employee emp && emp.FirstName.ToLower() == targetName.ToLower())
{
    Console.WriteLine($"Поточна посада: {emp.Position}. Введіть нову посаду: ");
    string newPos = Console.ReadLine() ?? "";
    emp.ChangePosition(newPos);
    Console.WriteLine("Посаду успішно змінено!");
    return;
}
}

Console.WriteLine("Працівника з таким прізвищем не знайдено.");
}

public static void IncreaseEmployeeSalary()
{
    Console.WriteLine("Введіть прізвище працівника: ");
    string targetName = Console.ReadLine() ?? "";

    foreach (Person p in database)
    {
        if (p is Employee emp && emp.FirstName.ToLower() == targetName.ToLower())
        {
            emp.IncreaseSalary();
            Console.WriteLine($"Оклад успішно збільшено! Новий оклад: {emp.Salary:F2} грн.");
            return;
        }
    }
    Console.WriteLine("Працівника з таким прізвищем не знайдено.");
}
}
}

```

### Лістинг Б.1 – Клас Menu, аркуш 4

```

using System;

namespace ConsoleApp1
{
    public class Person
    {
        private string firstName;
        private string lastName;
        private int height;
        private int weight;
        private string country;
        private string phoneNumber;

        public Person() : this("не вказано", "не вказано", 0, 0, "не вказано", "не вказано") { }

        public Person(string firstName, string lastName, int height, int weight, string country, string phoneNumber)
        {
            FirstName = firstName;
            LastName = lastName;
            Height = height;
            Weight = weight;
            Country = country;
            PhoneNumber = phoneNumber;
        }
    }
}

```

### Лістинг Б.2 – Клас Person, аркуш 1

```

CheckAndFixPhoneNumber();
}

public string FirstName
{
    get { return firstName; }
    set
    {
        if (string.IsNullOrEmpty(value))
        {
            throw new ArgumentException("Прізвище не може бути порожнім");
        }
        firstName = value;
    }
}

public string LastName
{
    get { return lastName; }
    set
    {
        if (string.IsNullOrEmpty(value))
        {
            throw new ArgumentException("Ім'я не може бути порожнім!");
        }
        lastName = value;
    }
}

public int Height
{
    get { return height; }
    set
    {
        if (value < 0 || value > 300)
        {
            throw new ArgumentException("Зріст повинен бути в діапазоні від 0 до 300 см");
        }
        height = value;
    }
}

public int Weight
{
    get { return weight; }
    set
    {
        if (value < 0)
        {
            throw new ArgumentException("Вага повинна бути більшою за 0");
        }
        weight = value;
    }
}

public string PhoneNumber
{
    get { return phoneNumber; }
    set
    {
        if (string.IsNullOrEmpty(value))
        {
            throw new ArgumentException("Номер телефону не може бути порожнім");
        }
    }
}

```

Лістинг Б.2 – Клас Person, аркуш 2

```

        phoneNumber = value;
    }
}

public string IsHeightOver200()
{
    if (height > 200)
    {
        return "Так";
    }
    else
    {
        return "Hi";
    }
}

public string LivesInPoland()
{
    string lowerCountry = country.ToLower();

    if (lowerCountry == "польща" || lowerCountry == "poland")
    {
        return "Так";
    }
    else
    {
        return "Hi";
    }
}

public void CheckAndFixPhoneNumber()
{
    if (!string.IsNullOrEmpty(phoneNumber) && phoneNumber.StartsWith("0"))
    {
        phoneNumber = "+38" + phoneNumber;
    }
}

public override string ToString()
{
    return $"[Людина] Прізвище: {FirstName}, Ім'я: {LastName}, Зріст: {Height}см, Вага: {Weight}кг, Країна: {Country}, Тел: {PhoneNumber}";
}
}
}

```

### Лістинг Б.2 – Клас Person, аркуш 3

```

using System;
using System.Collections.Generic;
using System.Text;

namespace ConsoleApp1
{
    public class Student : Person
    {
        private string university;
        private int courseYear;
        private string faculty;
        private List<int> grades;
    }
}

```

### Лістинг Б.3 – Клас Student, аркуш 1

```

public string University
{
    get { return university; }
    set { university = value; }
}

public int CourseYear
{
    get { return courseYear; }
    set { courseYear = value; }
}

public string Faculty
{
    get { return faculty; }
    set { faculty = value; }
}

public List<int> Grades
{
    get { return grades; }
    set { grades = value; }
}

public Student() : base()
{
    University = "не вказано";
    CourseYear = 1;
    Faculty = "не вказано";
    Grades = new List<int>();
}

public Student(string firstName, string lastName, int height, int weight, string country, string phoneNumber, string
university, int courseYear, string faculty, List<int> grades) : base(firstName, lastName, height, weight, country,
phoneNumber)
{
    University = university;
    CourseYear = courseYear;
    Faculty = faculty;

    if (grades == null)
    {
        Grades = new List<int>();
    }
    else
    {
        Grades = grades;
    }
}

public string GetScholarshipType()
{
    if (Grades.Count == 0) return "немає";

    bool allExcellent = true;
    foreach (int g in Grades)
    {
        if (g < 90)
        {
            allExcellent = false;
            break;
        }
    }
}
if (

```

Лістинг Б.3 – Клас Student, аркуш 2

```

allExcellent) return "підвищена";
    double sum = 0;
    foreach (int g in Grades)
    {
        sum += g;
    }
    double average = sum / Grades.Count;

```

### Лістинг Б.3 – Клас Student, аркуш 3

```

public string GetScholarshipType()
{
    if (Grades.Count == 0) return "немає";

    bool allExcellent = true;
    foreach (int g in Grades)
    {
        if (g < 90)
        {
            allExcellent = false;
            break;
        }
    }
    if (allExcellent) return "підвищена";

    double sum = 0;
    foreach (int g in Grades)
    {
        sum += g;
    }
    double average = sum / Grades.Count;

    if (average >= 75) return "звичайна";

    return "немає";
}

public override string ToString()
{
    return $"[Студент] Прізвище: {FirstName}, Ім'я: {LastName}, Зріст: {Height}см, ВНЗ: {University}, Курс: {CourseYear}, Факультет: {Faculty}, Оцінки: [{string.Join(", ", Grades)}], Стипендія: {GetScholarshipType()}";
}
}

```

### Лістинг Б.3 – Клас Student, аркуш 4

```

using System;
using System.Collections.Generic;
using System.Text;

namespace ConsoleApp1
{
    public class Employee : Person
    {
        private string company;
        private string position;
        private double salary;
        public string Company
        {
            get { return company; }
            set { company = value; }
        }

        public string Position
        {
            get { return position; }
            set { position = value; }
        }
        public double Salary
        {
            get { return salary; }
            set { salary = value; }
        }
        public Employee() : base()
        {
            Company = "не вказано";
            Position = "не вказано";
            Salary = 0.0;
        }

        public Employee(string firstName, string lastName, int height, int weight, string country, string phoneNumber,
            string company, string position, double salary) : base(firstName, lastName, height, weight, country, phoneNumber)
        {
            Company = company;
            Position = position;
            Salary = salary;
        }

        public void ChangePosition(string newPosition)
        {
            Position = newPosition;
        }

        public void IncreaseSalary()
        {
            string posLower = Position.ToLower();
            if (posLower.Contains("менеджер") || posLower.Contains("директор"))
            {
                Salary += Salary * 0.20;
            }
            Salary += Salary * 0.10;
        }
    }
    public override string ToString()
    {
        else if (posLower.Contains("розробник") || posLower.Contains("інженер"))
        {
            Salary += Salary * 0.15;
        }
    }
}

```

```
        else
        {
            return $"[Робітник] Прізвище: {FirstName}, Ім'я: {LastName}, Зріст: {Height}см, Країна: {Country}, Завод:
{Company}, Посада: {Position}, Оклад: {Salary:F2} грн";
        }
    }
}
```

Лістинг Б.4 – Клас Employee, аркуш 2